

Forecasting Monthly Electricity Revenue in Florida Using Seasonal Time Series Models

1. Introduction

Accurate forecasting of electricity revenue is essential for energy providers and policymakers because electricity demand exhibits strong seasonal patterns and is influenced by long-term economic and population trends. Reliable revenue forecasts support infrastructure planning, pricing strategies, and energy supply management.

This study develops statistical models to forecast monthly electricity revenue for the state of Florida using historical data obtained from the U.S. Energy Information Administration (EIA) in the form of two spreadsheets (1990-2009 and 2010-2025). The objective is to generate reliable forecasts for 2026 while evaluating multiple competing modelling strategies.

Several candidate forecasting approaches were evaluated using both Exponential Smoothing with Trend and Seasonality (ETS) and Seasonal Autoregressive Integrated Moving Average (SARIMA) models. Forecast performance was evaluated using a holdout validation design, with 2025 reserved as an out-of-sample test period. The model with the best holdout-period performance was then used to generate the final 2026 forecast.

The best-performing model was a SARIMA model with specification $ARIMA(3,1,0)(2,1,0)[12]$ applied to total direct electricity revenue beginning in 2010.

2. Data Description

The datasets contain monthly electricity sales and price information for each state, and filtering was used to single out Florida after combining the two time periods. While total revenue was reported, a manual calculation of revenue was obtained by multiplying electricity sales by sector-specific electricity prices.

The primary variable analyzed in this study is total monthly electricity revenue, measured in thousands of dollars.

Summary statistics for the total revenue series given in thousands of dollars indicate the following characteristics:

Statistic	Value
Start_Date	1/1/1990
End_Date	12/12/2025
N_Observations	432
Mean	2,080,668.27
Median	1,978,574.88
SD	464,931.03
Min	1,174,126.18
Q1	1,758,344.02
Q3	2,378,090.53

IQR	619,746.50
Max	3,618,351.17
Range	2,444,224.99
Skewness	0.807

These statistics demonstrate that electricity revenue varies substantially over time and exhibits large seasonal swings.

The long-run time series of total electricity revenue is shown below.

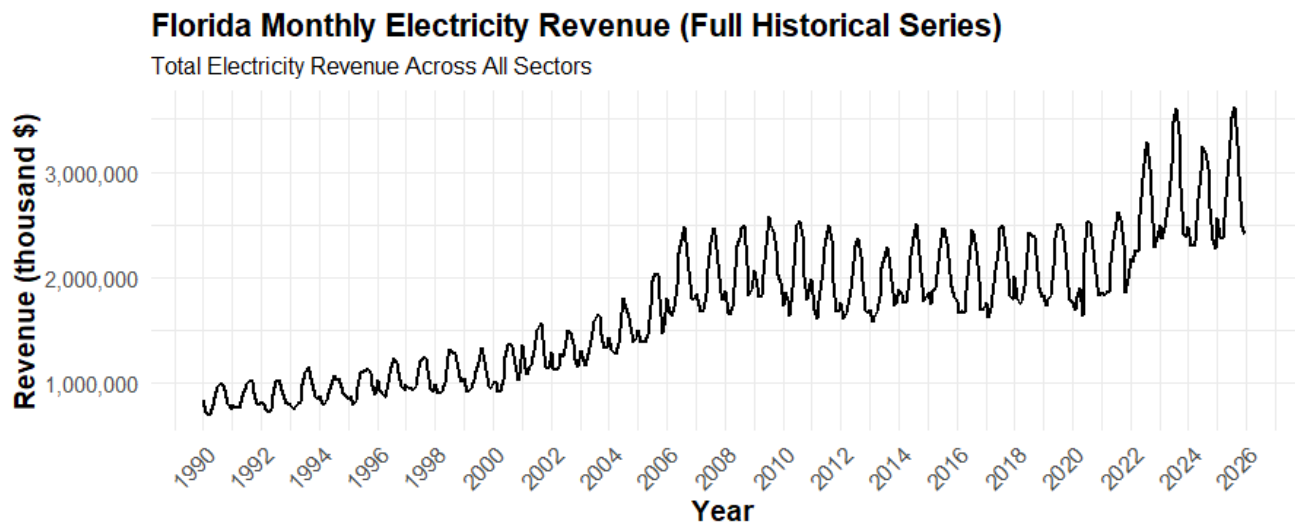


Figure 1 – Historical Total Electricity Revenue

The series shows a clear upward trend over time as well as strong seasonal fluctuations, with higher revenues during summer months when electricity demand increases, likely due to air-conditioning usage.

3. Data Cleaning and Validation

Several preprocessing steps were performed to ensure that the time series used for forecasting was reliable and internally consistent.

First, the raw datasets were combined and inspected for missing observations, inconsistent date formats, and duplicate entries. The “Other” sector was excluded from analysis because data were missing for more recent years, and the sector’s magnitude was small. The dataset contains monthly observations spanning January 1990 through December 2025, resulting in 432 observations. The time index was converted into a properly ordered monthly time series to ensure compatibility with forecasting methods.

Second, electricity revenue was reconstructed using the economic relationship:

$$\text{Revenue} = \text{Electricity Sales} \times \text{Electricity Price} / 100$$

This calculation was performed separately for each sector (residential, commercial, industrial, and transportation). Sector-level revenue values were then aggregated to produce a total revenue series. The division term is added to account for the units in the original datasets.

Reconstructing revenue allowed the reported revenue values to be validated against calculated values derived from the underlying economic variables. This step ensured that the revenue variable used in forecasting was internally consistent with the observed sales and price data.

Third, the calculated revenue values were compared to the reported revenue values to identify any discrepancies. Validation results indicated extremely small differences across all sectors, suggesting that the reported values are highly reliable. These small discrepancies likely arise from rounding differences or minor reporting adjustments.

Sector	Mean Absolute Difference	Max Percent Difference
Residential	226.89	0.064%
Commercial	172.52	0.078%
Industrial	36.95	0.103%
Transportation	0.17	0.086%
Total	311.80	0.047%

Finally, the revenue series was transformed into a monthly time series object suitable for time-series modelling. This allowed the application of seasonal forecasting models such as ETS and SARIMA.

4. Exploratory Time Series Analysis

Seasonal patterns in electricity revenue were examined using a seasonal plot.

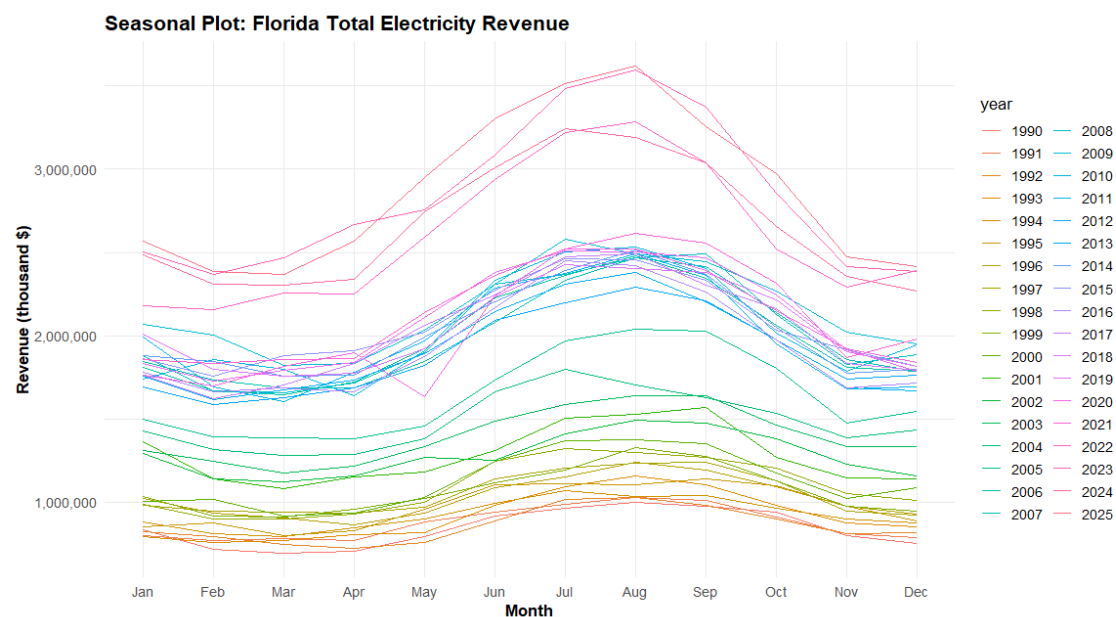


Figure 2 – Seasonal Pattern of Electricity Revenue

The seasonal plot confirms that electricity revenue follows a consistent seasonal pattern, with peaks typically occurring during summer months.

To further examine the series' structure, an STL decomposition was performed.

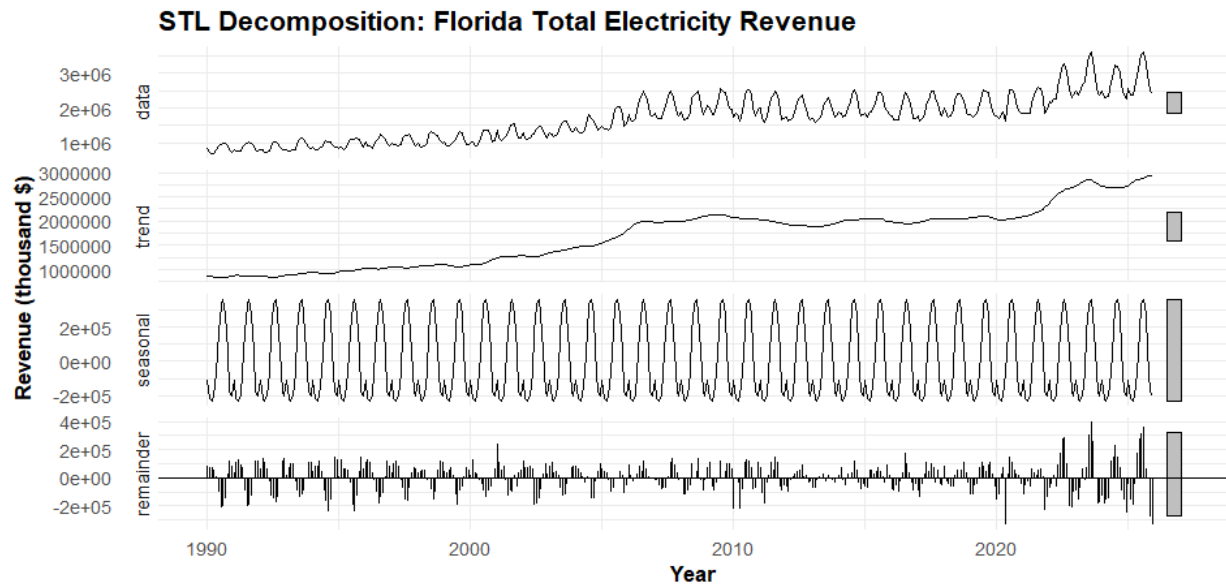


Figure 3 – STL Decomposition of Revenue Series

The decomposition separates the series into trend, seasonal, and irregular components. The trend component shows gradual long-term growth, while the seasonal component reflects stable yearly demand cycles.

These patterns suggest that models incorporating seasonal structure are appropriate for forecasting electricity revenue.

4.1 Variance Stabilization

The revenue series exhibits increasing variability as its level rises over time, suggesting heteroskedasticity. To stabilize the variance, a Box-Cox transformation was applied prior to model estimation. After forecasting, the predictions were back-transformed to the original revenue scale for interpretation.

4.2 Stationarity Assessment

Because ARIMA models require a stationary time series, the stationarity of the revenue series was assessed prior to model estimation using the ADF and KPSS tests. Visual inspection of the series suggested the presence of both a long-term trend and strong seasonality.

Differencing was therefore applied to remove these components. First-order differencing removed the long-term trend, while seasonal differencing with period 12 removed the annual seasonal pattern. After differencing, the series appeared stationary and suitable for ARIMA modelling.

5. Candidate Forecasting Models

Multiple candidate forecasting strategies were evaluated. Each strategy used a different method of constructing the revenue series or selecting the historical training window.

Candidate models included (ETS and SARIMA for all listed but Benchmark):

- Total direct revenue (full history)
- Total direct revenue (last 20 years)
- Total direct revenue (last 10 years)
- Total direct revenue beginning in 2010
- Sector revenue aggregates
- Sales $\times$ Price aggregate models
- Benchmark seasonal naïve model

In addition to ETS and SARIMA models, a seasonal naïve forecasting model was included as a benchmark. The seasonal naïve approach assumes that revenue in a given month will be equal to the revenue observed in the same month of the previous year. This model provides a useful baseline for evaluating whether more complex models provide meaningful improvements in forecast accuracy.

ETS models capture trend and seasonal patterns using exponential smoothing, while SARIMA models explicitly represent autoregressive relationships and seasonal dependencies in the time series.

6. Model Evaluation

Candidate specifications suggested by automated model selection procedures were evaluated and compared using holdout forecast accuracy. Forecast accuracy was evaluated using holdout validation, with 2025 excluded from model training and used as the out-of-sample evaluation period.

Forecast accuracy was measured using the following metrics:

- Root Mean Squared Error (RMSE)
- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)

Each metric was weighted equally, and models were scored based on their performance relative to others. Candidate SARIMA models were identified using ACF/PACF diagnostics and

automated model selection, and the final specification was selected based on the described holdout forecast accuracy.

The best candidate model was Total Direct – 2010+ with a SARIMA specification. See the table below for a comparison of this model with its ETS counterpart and the benchmark model. Appendix A shows comparisons of all candidates.

Model	RMSE	MSE	MAE	MAPE
SARIMA	112,877	12,741,318,000	93,416	3.20%
ETS	270,594	73,221,062,022	248,493	8.39%
Benchmark	230,243	53,011,949,955	201,010	6.80%

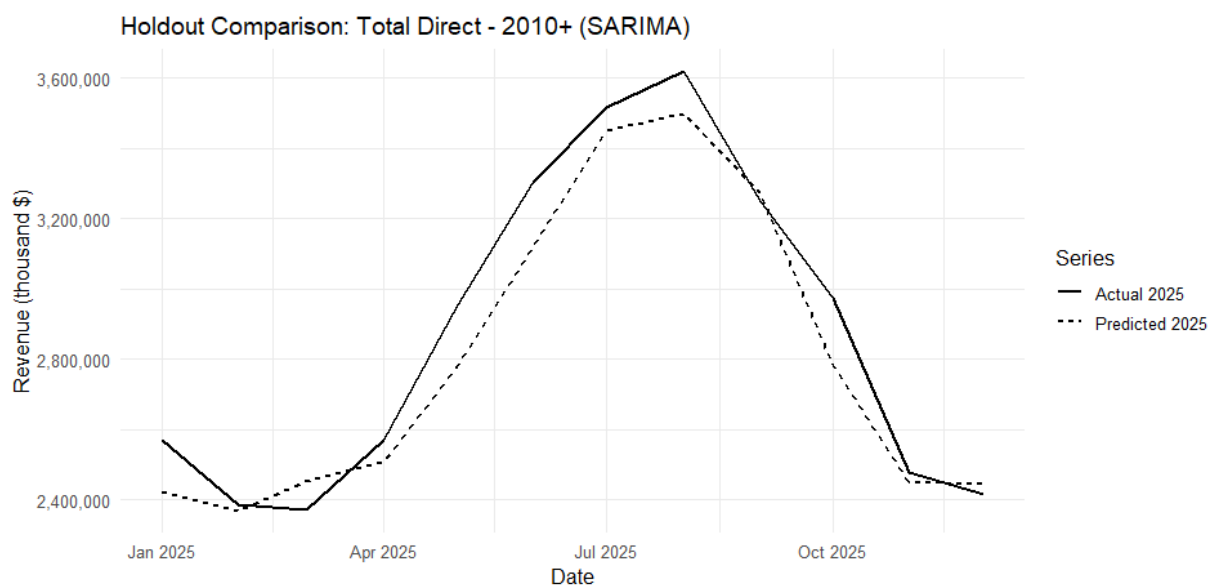


Figure 4 – Holdout Comparison

Figure 4 illustrates the performance of the selected model during the holdout evaluation period in 2025. The figure compares the model’s one-step-ahead forecasts with the observed revenue values.

The forecasts closely track observed seasonal fluctuations throughout the year, indicating that the model captures both the seasonal pattern and the short-term dynamics of electricity revenue. This visual comparison supports the statistical accuracy metrics reported above and confirms that the SARIMA model provides reliable out-of-sample forecasts.

Residual diagnostics also supported the adequacy of the SARIMA model. The Ljung-Box test produced a p-value of 0.1646, indicating no statistically significant autocorrelation in the residuals.

The SARIMA model likely outperformed ETS because the revenue series exhibits strong autoregressive structure in addition to seasonal dependence. SARIMA models explicitly capture lagged relationships among observations, thereby better representing the short-term dynamics observed in the revenue series.

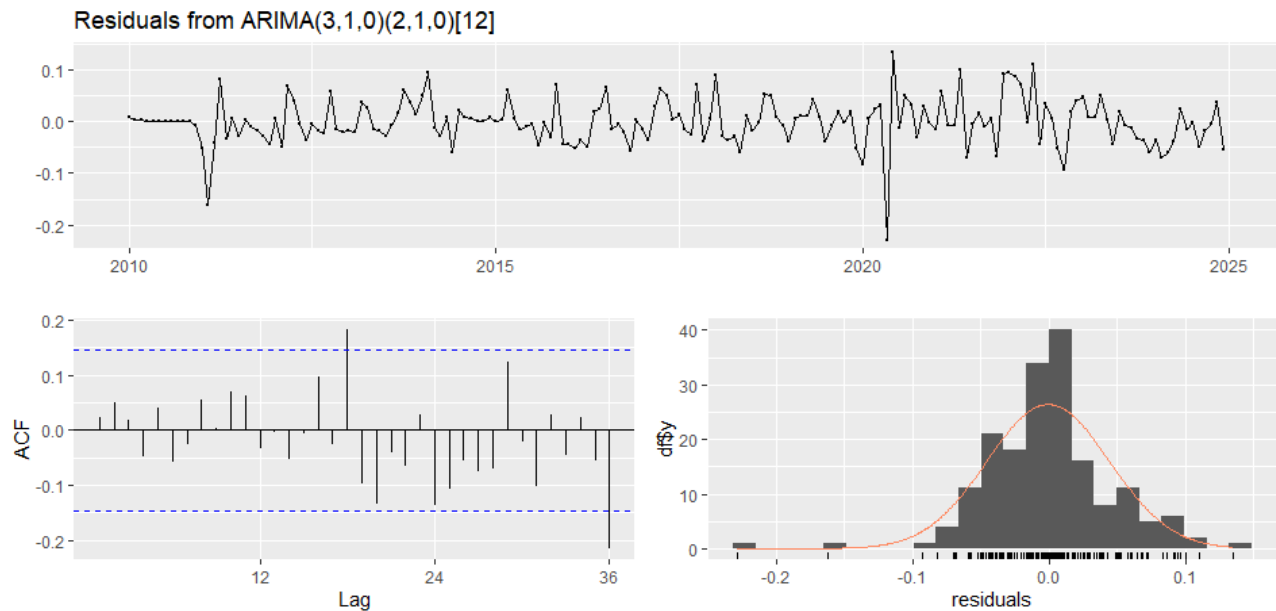


Figure 5 – Residual Diagnostics of Final SARIMA Model

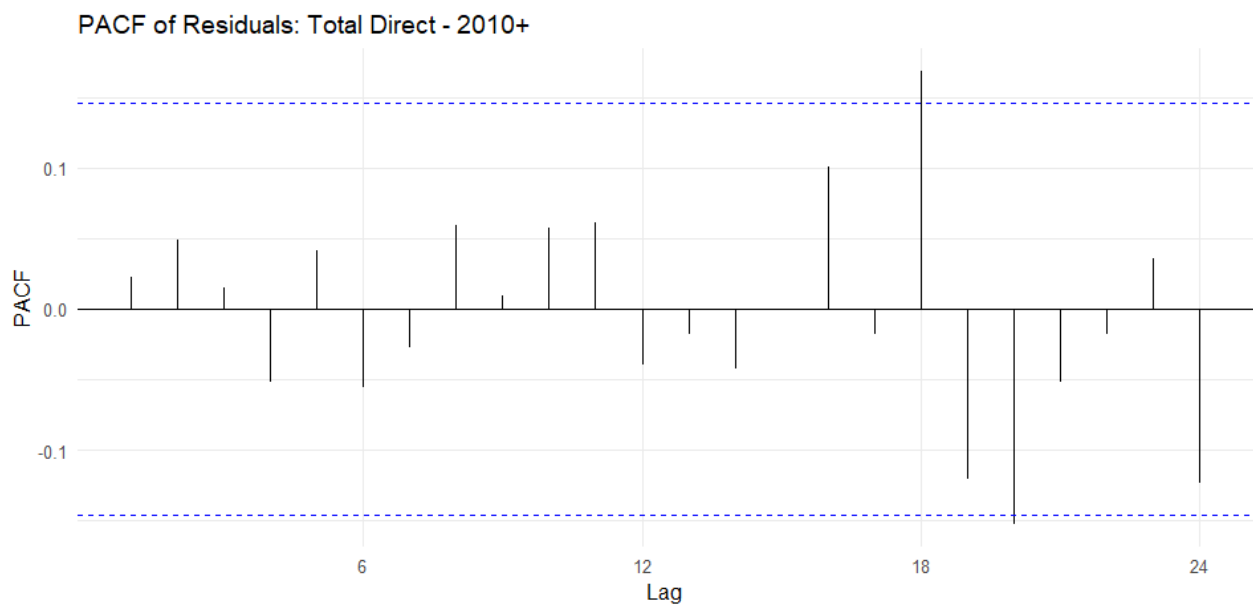


Figure 6 – PACF of Residuals

Figures 5 and 6 present the autocorrelation function (ACF) and partial autocorrelation function (PACF) of the model residuals. Most autocorrelation values fall within the 95% confidence

bounds, indicating that the residuals behave approximately like white noise. This suggests that the SARIMA model has successfully captured the underlying temporal dependence in the revenue series. The absence of systematic patterns in the residual autocorrelations is consistent with the Ljung-Box test results, which indicate no statistically significant remaining autocorrelation.

7. Final Model

The selected forecasting model was:

Candidate: Total Direct – 2010+

Strategy: Total Revenue Direct

Model Family: SARIMA

Specification: ARIMA(3,1,0)(2,1,0)[12]

The selected model includes the seasonal period of 12, which reflects the monthly frequency of the data and the presence of yearly seasonal patterns in electricity demand. The model includes three non-seasonal autoregressive terms, indicating that current revenue depends on values observed in recent months. A first difference and a seasonal difference were applied to remove long-term trends and annual seasonal effects, thereby stabilizing the series. The model also includes two seasonal autoregressive terms that capture relationships between observations separated by one and two years. Together, these components allow the model to capture both short-term dynamics and recurring seasonal patterns in electricity revenue.

8. Forecast Results for 2026

Using the selected SARIMA model, forecasts were generated for each month of 2026.

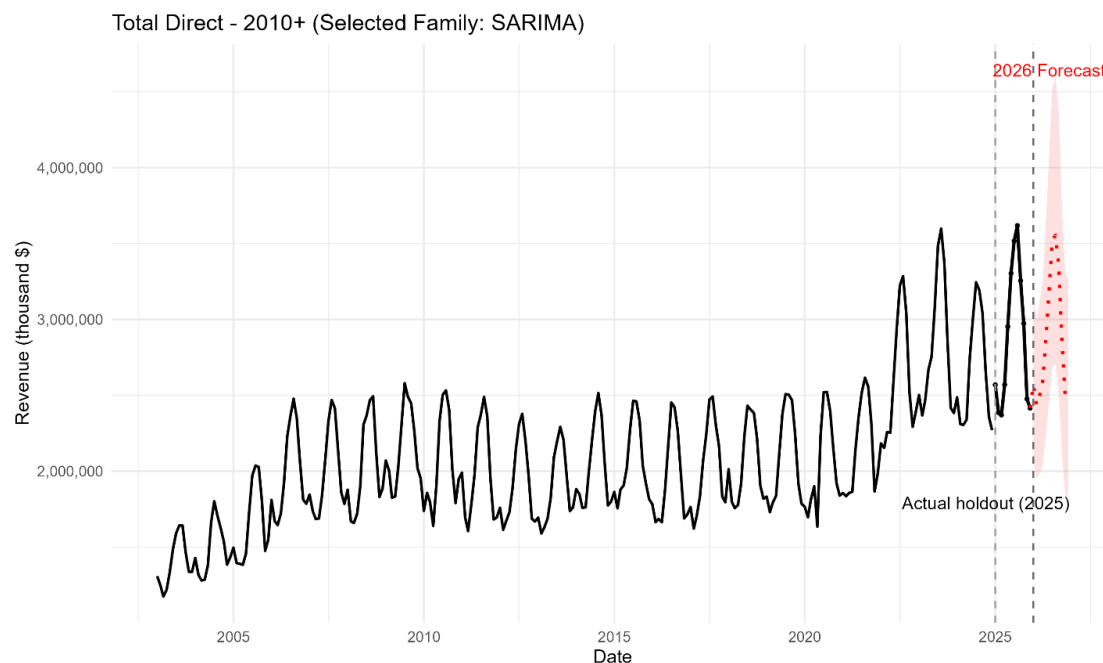


Figure 7 – Final SARIMA Forecast

The forecast maintains the strong seasonal structure observed in the historical data. The figure presented omits earlier historical data to ensure the graphic is interpretable. 95% prediction intervals were generated to quantify forecast uncertainty.

Monthly revenue forecasts for 2026 are shown below (thousands of dollars):

Month	Forecast	Lower	Upper
January	2,568,328	2,079,804	3,137,017
February	2,436,716	1,950,060	3,007,935
March	2,495,769	1,980,297	3,104,442
April	2,594,871	2,041,420	3,252,294
May	2,868,787	2,235,453	3,626,119
June	3,191,401	2,464,122	4,066,631
July	3,522,450	2,697,099	4,521,472
August	3,554,569	2,699,759	4,595,055
September	3,352,461	2,525,696	4,364,458
October	2,876,495	2,149,976	3,770,663
November	2,506,579	1,859,205	3,307,544
December	2,459,102	1,810,440	3,265,779

The forecast suggests that electricity revenue will peak in July and August, reaching approximately \$3.5 billion, before declining in the fall and winter months, similar to prior years.

9. Limitations

Although the selected SARIMA model performed well statistically, several limitations should be acknowledged.

First, electricity demand is highly sensitive to weather conditions such as extreme heat waves or unusually mild seasons. Because the model relies solely on historical revenue patterns and does not incorporate exogenous weather variables, unusual temperature patterns could cause forecast errors.

Second, electricity pricing policies and regulatory changes may alter revenue patterns in ways not captured by historical data. Structural changes in pricing frameworks or energy market regulations could therefore reduce forecast accuracy.

Third, the modelling process required several decisions regarding the appropriate training window and model specification. Electricity consumption patterns have evolved due to population growth, technological change, and improvements in energy efficiency. As a result, including very old observations may introduce structural shifts that reduce model performance,

while using shorter training windows may limit the amount of information available for parameter estimation.

Fourth, seasonal time series models such as SARIMA assume that seasonal patterns remain relatively stable over time. While historical data exhibit strong annual seasonality, future changes in climate patterns, improvements in energy efficiency, or shifts in electricity usage behaviour could alter these seasonal dynamics.

Finally, aggregated revenue models may mask sector-level dynamics. Different sectors may experience different growth patterns or structural changes. Modelling the aggregated series simplifies the forecasting problem but may overlook sector-specific variation that could influence future revenue patterns.

Future work could include weather-adjusted models to relate consumption patterns to weather patterns for additional insights. Furthermore, machine learning could be utilized to determine more concise predictions.

10. Conclusion

This study developed a seasonal time series forecasting model to predict monthly electricity revenue in Florida using historical data from the U.S. Energy Information Administration. Multiple candidate forecasting strategies were evaluated using out-of-sample holdout validation to ensure that model performance was assessed on unseen data.

The best-performing specification was a SARIMA(3,1,0)(2,1,0)[12] model applied to total electricity revenue beginning in 2010. During the holdout evaluation period, this model achieved a MAPE of 3.2%, indicating strong predictive accuracy relative to alternative modelling approaches, including exponential smoothing and a seasonal naïve benchmark.

The resulting forecasts suggest that electricity revenue in 2026 will continue to follow a pronounced seasonal pattern, with peak revenues occurring during the summer months, when cooling demand is highest. These forecasts also indicate continued stability in the underlying seasonal structure of electricity consumption in Florida.

Overall, the results demonstrate that seasonal time-series models provide an effective and interpretable approach for forecasting electricity revenue. Such models can support planning and decision-making within the energy sector by providing reliable short-term projections of revenue patterns and demand dynamics.

The full R script and key console outputs used to produce the results in this report are provided in the appendix for transparency and reproducibility.

Appendix A – Model Comparisons

Candidate	Start_ Date	Fam ily	Model_Descript ion	RMSE	MAE	MSE	MAPE	AICc
Total								-
Direct - 2010+	1/1/20 10	SAR IMA	ARIMA(3,1,0) (2 ,1,0) [12]	1128 77.4	9341 6.28	1.27 E+10	3.20 1338	531. 733
Total								-
Direct - Last 15 Years	1/1/20 10	SAR IMA	ARIMA(3,1,0) (2 ,1,0) [12]	1128 77.4	9341 6.28	1.27 E+10	3.20 1338	531. 733
Total								-
Direct - Last 10 Years	1/1/20 15	SAR IMA	ARIMA(3,1,0) (2 ,1,0) [12]	1182 67.8	9693 9.39	1.4E +10	3.29 1596	320. 096
Total								-
Direct - Last 20 Years	1/1/20 05	SAR IMA	ARIMA(3,0,0) (2 ,1,0) [12]	1300 16.3	1069 00.8	1.69 E+10	3.60 622	740. 103
Sector Revenue Aggregate - Last 10 Years	1/1/20 15	SAR IMA	Aggregate of sector-level SARIMA models	1407 27	1148 72.4	1.98 E+10	3.83 2839	NA
Sector Revenue Aggregate - 2010+	1/1/20 10	SAR IMA	Aggregate of sector-level SARIMA models	2155 58.1	1873 88.8	4.65 E+10	6.25 6163	NA
Sector Revenue Aggregate - Last 15 Years	1/1/20 10	SAR IMA	Aggregate of sector-level SARIMA models	2155 58.1	1873 88.8	4.65 E+10	6.25 6163	NA
Sales x Price Aggregate - 2010+	1/1/20 10	ETS	Aggregate of sector-level sales and price ETS models	2268 69.8	2064 76.4	5.15 E+10	6.95 2233	NA
Sales x Price Aggregate - Last 15 Years	1/1/20 10	ETS	Aggregate of sector-level sales and price ETS models	2268 69.8	2064 76.4	5.15 E+10	6.95 2233	NA
Benchmark Seasonal Naive	1/1/19 90	SNA IVE	Seasonal naive	2302 43.2	2040 40.2	5.3E +10	6.79 9457	NA
Total								-
Direct - Full	1/1/19 90	SAR IMA	ARIMA(1,0,2) (0 ,1,1) [12]	2497 24	2269 65.2	6.24 E+10	7.65 0534	1543 .38

History								
Total								
Direct -								
Last 10	1/1/20			2641	2400	6.98	8.07	149.
Years	15	ETS	ETS (A,Ad,A)	36.7	86.1	E+10	237	807
Total								
Direct -								
2010+	1/1/20			2705	2484	7.32	8.38	182.
	10	ETS	ETS (A,N,A)	93.9	93	E+10	5902	146
Total								
Direct -								
Last 15	1/1/20			2705	2484	7.32	8.38	182.
Years	10	ETS	ETS (A,N,A)	93.9	93	E+10	5902	146
Total								
Direct -								
Full	1/1/19			2739	2531	7.51	8.72	183.
History	90	ETS	ETS (A,Ad,A)	89.3	97.1	E+10	5868	893
Sector								
Revenue								
Aggregate	1/1/20		Aggregate of	2871	2624	8.25	8.82	
- 2010+	10	ETS	sector-level	57.1	28.5	E+10	9916	NA
Sector								
Revenue								
Aggregate			Aggregate of					
- Last 15	1/1/20		sector-level	2871	2624	8.25	8.82	
Years	10	ETS	ETS models	57.1	28.5	E+10	9916	NA
Total								
Direct -								
Last 20	1/1/20			2943	2678	8.66	9.01	180.
Years	05	ETS	ETS (A,Ad,A)	16.4	10.7	E+10	5491	403
Sector								
Revenue								
Aggregate			Aggregate of					
- Last 10	1/1/20		sector-level	3026	2769	9.16	9.31	
Years	15	ETS	ETS models	31.8	11.3	E+10	4826	NA
Sales x								
Price								
Aggregate	1/1/20		Aggregate of	3095	2912	9.58	9.89	
- 2010+	10	SAR	sector-level	53.7	26.4	E+10	6058	NA
		IMA	sales and					
Sales x								
Price								
Aggregate			Aggregate of					
- Last 15	1/1/20		sector-level	3095	2912	9.58	9.89	
Years	10	SAR	sales and	53.7	26.4	E+10	6058	NA
		IMA	price SARIMA					
models								

Appendix B – R Script

```
# BANA 5005 TERM PROJECT
# IMPORTANT UNITS:
#   Sales   = MWh
#   Price   = cents / kWh
# Revenue($ ) = Sales(MWh) * 1000(kWh/MWh) * Price(cents/kWh) / 100(cents/$)
```

```

#           = Sales * Price * 10
# Revenue(thousand $) = Revenue($) / 1000
#           = Sales * Price / 100

# 1) LOAD LIBRARIES
library(readxl)
library(dplyr)
library(tidyr)
library(lubridate)
library(forecast)
library(tseries)
library(ggplot2)
library(purrr)
library(tibble)
library(scales)

# 2) INPUTS
file_1990_2009 <- "C:/Users/14033/OneDrive - University of Lethbridge/Winter 2026/BANA
5005/HS861M 1990-2009.xlsx"
file_2010_2025 <- "C:/Users/14033/OneDrive - University of Lethbridge/Winter 2026/BANA
5005/HS861M 2010-.xlsx"
# Training ends Dec 2024
train_end_year <- 2024
train_end_month <- 12
# Holdout year = 2025
test_start_year <- 2025
test_start_month <- 1
test_end_year <- 2025
test_end_month <- 12
# Forecast 24 months from end of 2024:
# first 12 = 2025 holdout forecast
# last 12 = 2026 forecast
forecast_horizon <- 24
sectors <- c("Residential", "Commercial", "Industrial", "Transportation")

# 3) HELPER FUNCTIONS
# Build monthly Date from Year and Month
make_month_date <- function(year_vec, month_vec) {
  as.Date(sprintf("%04d-%02d-01", year_vec, month_vec))
}
# Clean monthly series:
# keep Date + selected value column
# create full monthly calendar through end_date
clean_monthly_series <- function(df, value_col, start_date = NULL, end_date = "2025-
12-01") {
  if (!(value_col %in% names(df))) {
    stop(paste("Column not found:", value_col))
  }
  out <- df %>%
    dplyr::select(Date, value = dplyr::all_of(value_col)) %>%
    dplyr::arrange(Date)
  if (!is.null(start_date)) {
    out <- out %>%
      dplyr::filter(Date >= as.Date(start_date))
  }
  out <- out %>%
    dplyr::filter(Date <= as.Date(end_date))
  full_dates <- tibble::tibble(
    Date = seq.Date(
      from = min(out$Date, na.rm = TRUE),
      to = as.Date(end_date),
      by = "month"
    )
  )
  out <- full_dates %>%
    dplyr::left_join(out, by = "Date") %>%

```

```

      dplyr::arrange(Date)
    out
  }
  # Convert cleaned data frame to monthly ts.
  make_ts_clean <- function(df, value_col = "value", allow_interpolation = TRUE) {
    if (!(value_col %in% names(df))) {
      stop(paste("Column not found in make_ts_clean:", value_col))
    }
    df <- df %>%
      dplyr::arrange(Date)
    if (nrow(df) < 36) {
      stop(paste("Not enough observations for", value_col))
    }
    if (max(df$Date, na.rm = TRUE) < as.Date("2025-12-01")) {
      stop(paste("Series does not extend through 2025-12 for", value_col))
    }
    x <- df[[value_col]]
    if (all(is.na(x))) {
      stop(paste("All values are NA for", value_col))
    }
    ts_obj <- stats::ts(
      x,
      start = c(lubridate::year(min(df$Date)), lubridate::month(min(df$Date))),
      frequency = 12
    )
    if (any(is.na(ts_obj))) {
      if (allow_interpolation) {
        ts_obj <- forecast::na.interp(ts_obj)
      } else {
        stop(paste("Missing values remain in", value_col))
      }
    }
    ts_obj
  }
  # Box-Cox lambda
  safe_lambda <- function(x) {
    x <- as.numeric(stats::na.omit(x))
    if (length(x) < 24) return(NULL)
    if (any(x <= 0)) return(NULL)
    tryCatch(
      forecast::BoxCox.lambda(x, lower = 0),
      error = function(e) NULL
    )
  }
  # Accuracy metrics on original scale
  calc_accuracy <- function(actual, predicted) {
    actual <- as.numeric(actual)
    predicted <- as.numeric(predicted)

    err <- actual - predicted
    tibble::tibble(
      RMSE = sqrt(mean(err^2, na.rm = TRUE)),
      MAE = mean(abs(err), na.rm = TRUE),
      MSE = mean(err^2, na.rm = TRUE),
      MAPE = mean(abs(err / actual), na.rm = TRUE) * 100
    )
  }
  # Add normalized weighted score within each candidate
  add_weighted_score <- function(df) {
    metric_cols <- c("RMSE", "MAE", "MSE", "MAPE")
    out <- df
    for (m in metric_cols) {
      x <- out[[m]]

```

```

    rng <- max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
    if (is.na(rng) || rng == 0) {
      out[[paste0("norm_", m)]] <- 0
    } else {
      out[[paste0("norm_", m)]] <- (x - min(x, na.rm = TRUE)) / rng
    }
  }
  out %>%
    dplyr::mutate(
      weighted_score =
        0.25 * norm_RMSE +
        0.25 * norm_MAE +
        0.25 * norm_MSE +
        0.25 * norm_MAPE
    )
}
# Compute start date from "last N years" relative to 2024
start_date_from_years_back <- function(years_back) {
  as.Date(sprintf("%04d-01-01", train_end_year - years_back + 1))
}
# Fit ETS or SARIMA to one univariate series
fit_one_family <- function(series_ts, family = c("ETS", "SARIMA")) {
  family <- match.arg(family)
  series_end <- tsp(series_ts)[2]
  required_test_end <- 2025 + 11/12
  if (series_end < required_test_end) {
    stop("Series does not extend far enough to create the 2025 holdout window.")
  }
  train_ts <- stats::window(series_ts, end = c(train_end_year, train_end_month))
  test_ts <- stats::window(series_ts,
    start = c(test_start_year, test_start_month),
    end = c(test_end_year, test_end_month))

  if (length(train_ts) < 36) {
    stop("Training series is too short after cleaning.")
  }

  if (length(test_ts) != 12) {
    stop("Test series is not exactly 12 months. Check date coverage.")
  }
  lambda_val <- safe_lambda(train_ts)

  if (family == "ETS") {
    fit <- forecast::ets(train_ts, lambda = lambda_val, biasadj = TRUE)
  } else {
    fit <- forecast::auto.arima(
      train_ts,
      seasonal = TRUE,
      stepwise = FALSE,
      approximation = FALSE,
      lambda = lambda_val,
      biasadj = TRUE
    )
  }

  fc <- forecast::forecast(fit, h = forecast_horizon, level = c(80, 95))

  fc_2025 <- as.numeric(fc$mean[1:12])
  fc_2026 <- as.numeric(fc$mean[13:24])
  # 95% intervals for 2026
  lower_2026 <- if (!is.null(fc$lower)) as.numeric(fc$lower[13:24, ncol(fc$lower)])
  else rep(NA_real_, 12)

```

```

upper_2026 <- if (!is.null(fc$upper)) as.numeric(fc$upper[13:24, ncol(fc$upper)])
else rep(NA_real_, 12)
acc <- calc_accuracy(as.numeric(test_ts), fc_2025)
aicc_val <- tryCatch(
  {
    if (!is.null(fit$aicc)) fit$aicc else forecast::AICc(fit)
  },
  error = function(e) NA_real_
)
list(
  family = family,
  fit = fit,
  lambda = lambda_val,
  train_ts = train_ts,
  test_ts = test_ts,
  forecast_2025 = fc_2025,
  forecast_2026 = fc_2026,
  lower_2026 = lower_2026,
  upper_2026 = upper_2026,
  accuracy = acc,
  AICc = aicc_val
)
}
# Plot full historical total revenue through 2025 and 2026
# forecast in red dotted line, connected from Dec 2025
plot_candidate_forecast <- function(full_df,
                                   forecast_2026,
                                   lower_2026 = NULL,
                                   upper_2026 = NULL,
                                   title_text) {

  full_df <- full_df %>%
    dplyr::arrange(Date)
  train_df <- full_df %>%
    dplyr::filter(Date <= as.Date("2024-12-01"))
  holdout_df <- full_df %>%
    dplyr::filter(Date >= as.Date("2025-01-01"),
                  Date <= as.Date("2025-12-01")) %>%
    dplyr::arrange(Date)
  forecast_dates <- seq.Date(as.Date("2026-01-01"), by = "month", length.out = 12)

  forecast_2026 <- as.numeric(unlist(forecast_2026, use.names = FALSE))
  if (length(forecast_2026) != 12) {
    stop("forecast_2026 must have length 12.")
  }
  if (!is.null(lower_2026)) {
    lower_2026 <- as.numeric(unlist(lower_2026, use.names = FALSE))
    if (length(lower_2026) != 12) stop("lower_2026 must have length 12.")
  }
  if (!is.null(upper_2026)) {
    upper_2026 <- as.numeric(unlist(upper_2026, use.names = FALSE))
    if (length(upper_2026) != 12) stop("upper_2026 must have length 12.")
  }
  if (nrow(holdout_df) == 0) {
    stop("Holdout data for 2025 is missing, so the forecast cannot be connected properly.")
  }
  last_actual_date <- max(holdout_df$Date)
  last_actual_value <- holdout_df %>%
    dplyr::filter(Date == last_actual_date) %>%
    dplyr::pull(value)

  forecast_line_df <- tibble::tibble(

```



```

    Date = c(last_actual_date, forecast_dates),
    Value = c(last_actual_value, forecast_2026)
  )
  if (!is.null(lower_2026) && !is.null(upper_2026)) {
    forecast_interval_df <- tibble::tibble(
      Date = forecast_dates,
      Forecast = forecast_2026,
      Lower = lower_2026,
      Upper = upper_2026
    )
  } else {
    forecast_interval_df <- NULL
  }
  ymax <- max(
    c(full_df$value,
      forecast_2026,
      if (!is.null(lower_2026)) lower_2026 else NA,
      if (!is.null(upper_2026)) upper_2026 else NA),
    na.rm = TRUE
  )
  ymin <- min(
    c(full_df$value,
      forecast_2026,
      if (!is.null(lower_2026)) lower_2026 else NA,
      if (!is.null(upper_2026)) upper_2026 else NA),
    na.rm = TRUE
  )
  p <- ggplot2::ggplot() +
    ggplot2::geom_line(
      data = train_df,
      ggplot2::aes(x = Date, y = value),
      color = "black",
      linewidth = 0.8
    ) +
    ggplot2::geom_line(
      data = holdout_df,
      ggplot2::aes(x = Date, y = value),
      color = "black",
      linewidth = 1
    ) +
    ggplot2::geom_point(
      data = holdout_df,
      ggplot2::aes(x = Date, y = value),
      color = "black",
      size = 0.8
    )
  if (!is.null(forecast_interval_df)) {
    p <- p +
      ggplot2::geom_ribbon(
        data = forecast_interval_df,
        ggplot2::aes(x = Date, ymin = Lower, ymax = Upper),
        inherit.aes = FALSE,
        fill = "red",
        alpha = 0.12
      )
  }
  p <- p +
    ggplot2::geom_line(
      data = forecast_line_df,
      ggplot2::aes(x = Date, y = Value),
      color = "red",
      linewidth = 1,
      linetype = "dotted"
    )

```

```

    ) +
    ggplot2::geom_vline(
      xintercept = as.numeric(as.Date("2025-01-01")),
      linetype = "dashed",
      color = "grey60"
    ) +
    ggplot2::geom_vline(
      xintercept = as.numeric(as.Date("2026-01-01")),
      linetype = "dashed",
      color = "grey40"
    ) +
    ggplot2::annotate(
      "text",
      x = as.Date("2024-10-01"),
      y = ymin + 0.18 * (ymax - ymin),
      label = "Actual holdout (2025)",
      size = 4
    ) +
    ggplot2::annotate(
      "text",
      x = as.Date("2026-06-01"),
      y = ymax * 1.01,
      label = "2026 Forecast",
      color = "red",
      size = 4
    ) +
    ggplot2::scale_y_continuous(labels = scales::comma) +
    ggplot2::labs(
      title = title_text,
      x = "Date",
      y = "Revenue (thousand $)"
    ) +
    ggplot2::theme_minimal(base_size = 12)
  return(p)
}

# 4) IMPORT DATA
data_1990_2009 <- readxl::read_excel("C:/Users/14033/OneDrive - University of
Lethbridge/Winter 2026/BANA 5005/HS861M 1990-2009.xlsx")
data_2010_2025 <- readxl::read_excel("C:/Users/14033/OneDrive - University of
Lethbridge/Winter 2026/BANA 5005/HS861M 2010-.xlsx")
data_all <- dplyr::bind_rows(data_1990_2009, data_2010_2025) %>%
  dplyr::mutate(Date = make_month_date(Year, Month)) %>%
  dplyr::arrange(Date)

# 5) FILTER FLORIDA
data_fl <- data_all %>%
  dplyr::filter(State == "FL") %>%
  dplyr::arrange(Date)

# Optional diagnostic: inspect status values instead of dropping them blindly
if ("Data Status" %in% names(data_fl)) {
  cat("\n===== FLORIDA DATA STATUS COUNTS =====\n")
  print(data_fl %>% dplyr::count(`Data Status`, sort = TRUE))
}

if ("Data.Status" %in% names(data_fl)) {
  cat("\n===== FLORIDA DATA.STATUS COUNTS =====\n")
  print(data_fl %>% dplyr::count(Data.Status, sort = TRUE))
}

# 6) VALIDATE REQUIRED COLUMNS
required_cols <- c(
  "Year", "Month", "State", "Date",
  "Residential_Sales", "Residential_Price",
  "Commercial_Sales", "Commercial_Price",
  "Industrial_Sales", "Industrial_Price",
  "Transportation_Sales", "Transportation_Price"
)

```

```

)
missing_required <- setdiff(required_cols, names(data_fl))
if (length(missing_required) > 0) {
  stop(paste("Missing required columns:", paste(missing_required, collapse = ", ")))
}
# 7) CALCULATE REVENUE IN THOUSANDS OF DOLLARS
# Revenue(thousand $) = Sales * Price / 100
for (s in sectors) {
  sales_col <- paste0(s, "_Sales")
  price_col <- paste0(s, "_Price")
  rev_col <- paste0(s, "_Revenue_Calc")

  data_fl[[rev_col]] <- data_fl[[sales_col]] * data_fl[[price_col]] / 100
}
data_fl <- data_fl %>%
  dplyr::mutate(
    Total_Revenue_Calc =
      Residential_Revenue_Calc +
      Commercial_Revenue_Calc +
      Industrial_Revenue_Calc +
      Transportation_Revenue_Calc
  )
# 8) MISSING VALUE CHECKS
cat("\n===== MISSING VALUE COUNTS =====\n")
print(colSums(is.na(data_fl)))
cat("\n===== ROWS WITH MISSING REVENUE VALUES =====\n")
print(
  data_fl %>%
    dplyr::select(Date, dplyr::ends_with("_Revenue_Calc"), Total_Revenue_Calc) %>%
    dplyr::filter(dplyr::if_any(-Date, is.na))
)
# 9) SUMMARY STATS FOR REPORT
summary_stats_total <- data_fl %>%
  dplyr::summarise(
    Series = "Florida Total Electricity Revenue",
    Start_Date = min(Date, na.rm = TRUE),
    End_Date = max(Date, na.rm = TRUE),
    N_Observations = dplyr::n(),
    N_Nonmissing = sum(!is.na(Total_Revenue_Calc)),
    N_Missing = sum(is.na(Total_Revenue_Calc)),
    Mean = mean(Total_Revenue_Calc, na.rm = TRUE),
    Median = stats::median(Total_Revenue_Calc, na.rm = TRUE),
    SD = stats::sd(Total_Revenue_Calc, na.rm = TRUE),
    Variance = stats::var(Total_Revenue_Calc, na.rm = TRUE),
    Min = min(Total_Revenue_Calc, na.rm = TRUE),
    Q1 = as.numeric(stats::quantile(Total_Revenue_Calc, 0.25, na.rm = TRUE)),
    Q3 = as.numeric(stats::quantile(Total_Revenue_Calc, 0.75, na.rm = TRUE)),
    IQR = stats::IQR(Total_Revenue_Calc, na.rm = TRUE),
    Max = max(Total_Revenue_Calc, na.rm = TRUE),
    Range = max(Total_Revenue_Calc, na.rm = TRUE) - min(Total_Revenue_Calc, na.rm =
TRUE),
    CV = stats::sd(Total_Revenue_Calc, na.rm = TRUE) / mean(Total_Revenue_Calc, na.rm
= TRUE),
    Skewness = mean((Total_Revenue_Calc - mean(Total_Revenue_Calc, na.rm = TRUE))^3,
na.rm = TRUE) /
      (stats::sd(Total_Revenue_Calc, na.rm = TRUE)^3)
  )
print(summary_stats_total)
# LONG FORMAT TABLE FOR REPORT
descriptive_statistics_table <- summary_stats_total %>%
  dplyr::mutate(across(-Series, as.character)) %>% # convert to same type
  tidyr::pivot_longer(
    cols = -Series,

```

```

        names_to = "Statistic",
        values_to = "Value"
    )
print(descriptive_statistics_table)
# 10) DIAGNOSTIC PLOTS AND STATIONARITY CHECKS FOR REPORT
total_full_df_for_diag <- clean_monthly_series(data_fl, "Total_Revenue_Calc")
ts_total_full_for_diag <- make_ts_clean(total_full_df_for_diag, "value")
forecast::autoplot(ts_total_full_for_diag) +
  ggplot2::labs(
    title = "Florida Total Revenue (Historical)",
    x = "Year",
    y = "Revenue (thousand $)"
  )
forecast::ggseasonplot(ts_total_full_for_diag) +
  ggplot2::labs(
    title = "Seasonal Plot: Florida Total Revenue",
    y = "Revenue (thousand $)"
  )
forecast::ggsubseriesplot(ts_total_full_for_diag) +
  ggplot2::labs(
    title = "Seasonal Subseries Plot: Florida Total Revenue",
    y = "Revenue (thousand $)"
  )
decomp_total <- stats::stl(ts_total_full_for_diag, s.window = "periodic")
forecast::autoplot(decomp_total) +
  ggplot2::labs(title = "STL Decomposition: Florida Total Revenue")
forecast::ggAcf(ts_total_full_for_diag) +
  ggplot2::labs(title = "ACF: Florida Total Revenue")
forecast::ggPacf(ts_total_full_for_diag) +
  ggplot2::labs(title = "PACF: Florida Total Revenue")
lambda_diag <- safe_lambda(ts_total_full_for_diag)
ts_total_for_tests <- if (!is.null(lambda_diag))
forecast::BoxCox(ts_total_full_for_diag, lambda_diag) else ts_total_full_for_diag
cat("\n===== STATIONARITY TESTS (DIAGNOSTIC SERIES) =====\n")
print(tseries::adf.test(ts_total_for_tests))
print(tseries::kpss.test(ts_total_for_tests))
cat("\nSuggested regular differences:", forecast::ndiffs(ts_total_for_tests), "\n")
cat("\nSuggested seasonal differences:", forecast::nsdiffs(ts_total_for_tests), "\n")
# REPLACEMENT FOR SECTIONS 11-23
# Implements:
#   - benchmark seasonal naive model
#   - revenue reconciliation checks
#   - model description extraction
#   - numeric residual diagnostics
#   - approximate aggregate forecast intervals
#   - candidate summary export table
#   - improved full-history plots
#   - holdout evaluation plots
#   - limitations/diagnostic summary
# 11) OPTIONAL REVENUE RECONCILIATION CHECKS
cat("\n===== REVENUE RECONCILIATION CHECKS =====\n")
# Revenue validation: reported vs calculated
validation_sectors <- c(sectors, "Total")
revenue_validation_results <- purrr::map_dfr(validation_sectors, function(s) {

  if (s == "Total") {
    reported_col <- "Total_Revenue"
    calc_col <- "Total_Revenue_Calc"
  } else {
    reported_col <- paste0(s, "_Revenue")
    calc_col <- paste0(s, "_Revenue_Calc")
  }

```

```

if (reported_col %in% names(data_fl)) {

  reported <- data_fl[[reported_col]]
  calc      <- data_fl[[calc_col]]

  abs_diff <- abs(reported - calc)

  pct_diff <- abs_diff / reported * 100

  tibble::tibble(
    Sector = s,
    Mean_Abs_Diff = mean(abs_diff, na.rm = TRUE),
    Max_Abs_Diff  = max(abs_diff, na.rm = TRUE),
    Mean_Pct_Diff = mean(pct_diff, na.rm = TRUE),
    Max_Pct_Diff  = max(pct_diff, na.rm = TRUE),
    N_Compared    = sum(!is.na(reported) & !is.na(calc))
  )

} else {

  tibble::tibble(
    Sector = s,
    Mean_Abs_Diff = NA_real_,
    Max_Abs_Diff  = NA_real_,
    Mean_Pct_Diff = NA_real_,
    Max_Pct_Diff  = NA_real_,
    N_Compared    = 0
  )
}
})
print(revenue_validation_results)
# 12) EXTRA HELPER FUNCTIONS
# Extract model description for report
extract_model_desc <- function(fit, family) {
  if (is.null(fit)) return(NA_character_)

  if (family == "ETS") {
    out <- tryCatch(fit$method, error = function(e) NA_character_)
    return(out)
  }
  if (family == "SARIMA") {
    ord <- tryCatch(forecast::arimaorder(fit), error = function(e) NULL)
    if (is.null(ord)) return(NA_character_)
    return(
      sprintf(
        "ARIMA(%d,%d,%d) (%d,%d,%d) [%d]",
        ord[1], ord[2], ord[3], ord[4], ord[5], ord[6], ord[7]
      )
    )
  }
}

NA_character_
}
# Numeric residual diagnostics for direct models
extract_residual_diagnostics <- function(fit) {
  if (is.null(fit)) {
    return(
      tibble::tibble(
        Resid_Mean = NA_real_,
        Resid_SD   = NA_real_,
        Ljung_Box_pvalue = NA_real_,
        Resid_ACF12 = NA_real_
      )
    )
  }
}

```

```

    )
  }
  res <- tryCatch(as.numeric(stats::residuals(fit)), error = function(e) NULL)
  if (is.null(res) || length(stats::na.omit(res)) < 24) {
    return(
      tibble::tibble(
        Resid_Mean = NA_real_,
        Resid_SD = NA_real_,
        Ljung_Box_pvalue = NA_real_,
        Resid_ACF12 = NA_real_
      )
    )
  }

  lb_p <- tryCatch(
    stats::Box.test(res, lag = 24, type = "Ljung-Box")$p.value,
    error = function(e) NA_real_
  )
  acf12 <- tryCatch(
    {
      ac <- stats::acf(res, plot = FALSE, lag.max = 12, na.action = na.pass)
      as.numeric(ac$acf[13])
    },
    error = function(e) NA_real_
  )
  tibble::tibble(
    Resid_Mean = mean(res, na.rm = TRUE),
    Resid_SD = stats::sd(res, na.rm = TRUE),
    Ljung_Box_pvalue = lb_p,
    Resid_ACF12 = acf12
  )
}
# Approximate aggregate forecast interval by summing variances
# For sales*price strategy, this remains an approximation.
combine_component_intervals <- function(mean_list, lower_list, upper_list, level_z =
1.96) {
  mean_mat <- do.call(cbind, mean_list)
  lower_mat <- do.call(cbind, lower_list)
  upper_mat <- do.call(cbind, upper_list)
  agg_mean <- rowSums(mean_mat, na.rm = TRUE)
  # approximate sd from symmetric interval width
  comp_sd <- (upper_mat - lower_mat) / (2 * level_z)
  comp_var <- comp_sd^2
  agg_sd <- sqrt(rowSums(comp_var, na.rm = TRUE))
  agg_lower <- agg_mean - level_z * agg_sd
  agg_upper <- agg_mean + level_z * agg_sd
  list(
    mean = agg_mean,
    lower = agg_lower,
    upper = agg_upper
  )
}
# Benchmark seasonal naive
fit_benchmark_candidate <- function(series_ts, candidate_name = "Benchmark Seasonal
Naive") {
  train_ts <- stats::window(series_ts, end = c(train_end_year, train_end_month))
  test_ts <- stats::window(
    series_ts,
    start = c(test_start_year, test_start_month),
    end = c(test_end_year, test_end_month)
  )
  fit <- forecast::snaive(train_ts, h = forecast_horizon, level = c(80, 95))
  fc_2025 <- as.numeric(fit$mean[1:12])

```

```

    fc_2026 <- as.numeric(fit$mean[13:24])
    lower_2026 <- if (!is.null(fit$lower)) as.numeric(fit$lower[13:24, ncol(fit$lower)])
  else rep(NA_real_, 12)
    upper_2026 <- if (!is.null(fit$upper)) as.numeric(fit$upper[13:24, ncol(fit$upper)])
  else rep(NA_real_, 12)
    acc <- calc_accuracy(as.numeric(test_ts), fc_2025)
    tibble::tibble(
      Candidate = candidate_name,
      Strategy = "Benchmark",
      Start_Date = as.Date(sprintf("%04d-%02d-01", start(train_ts)[1],
start(train_ts)[2])),
      Family = "SNAIVE",
      RMSE = acc$RMSE,
      MAE = acc$MAE,
      MSE = acc$MSE,
      MAPE = acc$MAPE,
      AICc = NA_real_,
      Forecast_2025 = list(fc_2025),
      Forecast_2026 = list(fc_2026),
      Lower_2026_95 = list(lower_2026),
      Upper_2026_95 = list(upper_2026),
      Fit_Object = list(fit),
      Model_Description = "Seasonal naive",
      Lambda = NA_real_,
      Resid_Mean = NA_real_,
      Resid_SD = NA_real_,
      Ljung_Box_pvalue = NA_real_,
      Resid_ACF12 = NA_real_
    )
  }
# Plot full historical total revenue through 2025 and 2026
# with 2025 shown as actual holdout and 2026 as red dotted forecast
plot_candidate_forecast <- function(full_df, forecast_2026, title_text) {
  train_df <- full_df %>%
    dplyr::filter(Date <= as.Date("2024-12-01"))

  holdout_df <- full_df %>%
    dplyr::filter(Date >= as.Date("2025-01-01"),
      Date <= as.Date("2025-12-01"))

  forecast_dates <- seq.Date(as.Date("2026-01-01"), by = "month", length.out = 12)

  last_actual_2025 <- full_df %>%
    dplyr::filter(Date == as.Date("2025-12-01")) %>%
    dplyr::pull(value)

  forecast_df <- tibble::tibble(
    Date = c(as.Date("2025-12-01"), forecast_dates),
    Value = c(last_actual_2025, as.numeric(forecast_2026))
  )
  ymax <- max(c(full_df$value, as.numeric(forecast_2026)), na.rm = TRUE)
  ggplot2::ggplot() +
    ggplot2::geom_line(
      data = train_df,
      ggplot2::aes(x = Date, y = value),
      color = "black",
      linewidth = 0.8
    ) +
    ggplot2::geom_line(
      data = holdout_df,
      ggplot2::aes(x = Date, y = value),
      color = "black",
      linewidth = 1
    )

```

```

) +
ggplot2::geom_point(
  data = holdout_df,
  ggplot2::aes(x = Date, y = value),
  color = "black",
  size = 0.7
) +
ggplot2::geom_line(
  data = forecast_df,
  ggplot2::aes(x = Date, y = Value),
  color = "red",
  linewidth = 1,
  linetype = "dotted"
) +
ggplot2::geom_vline(
  xintercept = as.numeric(as.Date("2025-01-01")),
  linetype = "dashed",
  color = "grey60"
) +
ggplot2::geom_vline(
  xintercept = as.numeric(as.Date("2026-01-01")),
  linetype = "dashed",
  color = "grey40"
) +
ggplot2::annotate(
  "text",
  x = as.Date("2025-06-01"),
  y = ymax * 0.4,
  label = "Actual holdout (2025)",
  vjust = -0.3,
  size = 3
) +
ggplot2::annotate(
  "text",
  x = as.Date("2026-06-01"),
  y = ymax * 1.03,
  label = "Forecast (2026)",
  color = "red",
  vjust = -0.3,
  size = 3
) +
ggplot2::scale_y_continuous(labels = scales::comma) +
ggplot2::labs(
  title = title_text,
  x = "Date",
  y = "Revenue (thousand $)"
) +
ggplot2::theme_minimal(base_size = 12)
}
# Holdout comparison plot: actual 2025 vs predicted 2025
plot_holdout_comparison <- function(actual_2025, predicted_2025, candidate, family) {
  holdout_dates <- seq.Date(as.Date("2025-01-01"), by = "month", length.out = 12)

  df_plot <- tibble::tibble(
    Date = rep(holdout_dates, 2),
    Value = c(as.numeric(actual_2025), as.numeric(predicted_2025)),
    Series = rep(c("Actual 2025", "Predicted 2025"), each = 12)
  )

  ggplot2::ggplot(df_plot, ggplot2::aes(x = Date, y = Value, linetype = Series)) +
    ggplot2::geom_line(linewidth = 1, color = "black") +
    ggplot2::scale_y_continuous(labels = scales::comma) +
    ggplot2::labs(

```



```

        title = paste0("Holdout Comparison: ", candidate, " (", family, ")"),
        x = "Date",
        y = "Revenue (thousand $)"
    ) +
    ggplot2::theme_minimal(base_size = 12)
}
# 13) CANDIDATE MODEL DEFINITIONS
candidate_definitions <- tibble::tibble(
  candidate_id = 1:10,
  candidate_name = c(
    "Total Direct - Full History",
    "Total Direct - Last 20 Years",
    "Total Direct - Last 15 Years",
    "Total Direct - Last 10 Years",
    "Total Direct - 2010+",
    "Sector Revenue Aggregate - 2010+",
    "Sector Revenue Aggregate - Last 15 Years",
    "Sector Revenue Aggregate - Last 10 Years",
    "Sales x Price Aggregate - 2010+",
    "Sales x Price Aggregate - Last 15 Years"
  ),
  strategy = c(
    "total_direct",
    "total_direct",
    "total_direct",
    "total_direct",
    "total_direct",
    "sector_revenue_aggregate",
    "sector_revenue_aggregate",
    "sector_revenue_aggregate",
    "sales_price_aggregate",
    "sales_price_aggregate"
  ),
  start_date = c(
    min(data_fl$Date, na.rm = TRUE),
    start_date_from_years_back(20),
    start_date_from_years_back(15),
    start_date_from_years_back(10),
    as.Date("2010-01-01"),
    as.Date("2010-01-01"),
    start_date_from_years_back(15),
    start_date_from_years_back(10),
    as.Date("2010-01-01"),
    start_date_from_years_back(15)
  )
) %>%
  dplyr::mutate(
    Families_Test = "ETS, SARIMA"
  )
print(candidate_definitions)
# 14) FIT FUNCTIONS FOR EACH STRATEGY
# Strategy A: model total revenue directly
fit_total_direct_candidate <- function(df_in, start_date, family, candidate_name) {
  total_df <- clean_monthly_series(df_in, "Total Revenue Calc", start_date =
start_date)
  total_ts <- make_ts_clean(total_df, "value")
  fit_res <- fit_one_family(total_ts, family = family)
  diag_res <- extract_residual_diagnostics(fit_res$fit)
  tibble::tibble(
    Candidate = candidate_name,
    Strategy = "Total Revenue Direct",
    Start_Date = as.Date(start_date),
    Family = family,

```

```

    RMSE = fit_res$accuracy$RMSE,
    MAE = fit_res$accuracy$MAE,
    MSE = fit_res$accuracy$MSE,
    MAPE = fit_res$accuracy$MAPE,
    AICc = fit_res$AICc,
    Forecast_2025 = list(fit_res$forecast_2025),
    Forecast_2026 = list(fit_res$forecast_2026),
    Lower_2026_95 = list(fit_res$lower_2026),
    Upper_2026_95 = list(fit_res$upper_2026),
    Fit_Object = list(fit_res$fit),
    Model_Description = extract_model_desc(fit_res$fit, family),
    Lambda = fit_res$lambda,
    Resid_Mean = diag_res$Resid_Mean,
    Resid_SD = diag_res$Resid_SD,
    Ljung_Box_pvalue = diag_res$Ljung_Box_pvalue,
    Resid_ACF12 = diag_res$Resid_ACF12
  )
}
# Strategy B: model each sector revenue directly, then aggregate
fit_sector_revenue_candidate <- function(df_in, start_date, family, candidate_name) {
  df_sub <- df_in %>%
    dplyr::filter(Date >= as.Date(start_date)) %>%
    dplyr::arrange(Date)

  total_df <- clean_monthly_series(df_sub, "Total_Revenue_Calc")
  total_ts <- make_ts_clean(total_df, "value")
  actual_2025 <- stats::window(total_ts, start = c(2025, 1), end = c(2025, 12))

  sector_results <- purrr::map(sectors, function(s) {
    rev_col <- paste0(s, "_Revenue_Calc")
    sector_df <- clean_monthly_series(df_sub, rev_col)
    sector_ts <- make_ts_clean(sector_df, "value")
    fit_one_family(sector_ts, family = family)
  })
  agg_2025 <- Reduce(`+`, purrr::map(sector_results, "forecast_2025"))
  agg_2026 <- Reduce(`+`, purrr::map(sector_results, "forecast_2026"))

  interval_res <- combine_component_intervals(
    mean_list = purrr::map(sector_results, "forecast_2026"),
    lower_list = purrr::map(sector_results, "lower_2026"),
    upper_list = purrr::map(sector_results, "upper_2026")
  )
  acc <- calc_accuracy(actual_2025, agg_2025)
  tibble::tibble(
    Candidate = candidate_name,
    Strategy = "Sector Revenue Then Aggregate",
    Start_Date = as.Date(start_date),
    Family = family,
    RMSE = acc$RMSE,
    MAE = acc$MAE,
    MSE = acc$MSE,
    MAPE = acc$MAPE,
    AICc = NA_real_,
    Forecast_2025 = list(agg_2025),
    Forecast_2026 = list(interval_res$mean),
    Lower_2026_95 = list(interval_res$lower),
    Upper_2026_95 = list(interval_res$upper),
    Fit_Object = list(NULL),
    Model_Description = paste0("Aggregate of sector-level ", family, " models"),
    Lambda = NA_real_,
    Resid_Mean = NA_real_,
    Resid_SD = NA_real_,
    Ljung_Box_pvalue = NA_real_,

```

```

    Resid_ACF12 = NA_real_
  )
}
# Strategy C: model sales and price separately, then aggregate
fit_sales_price_candidate <- function(df_in, start_date, family, candidate_name) {
  df_sub <- df_in %>%
    dplyr::filter(Date >= as.Date(start_date)) %>%
    dplyr::arrange(Date)
  total_df <- clean_monthly_series(df_sub, "Total_Revenue_Calc")
  total_ts <- make_ts_clean(total_df, "value")
  actual_2025 <- stats::window(total_ts, start = c(2025, 1), end = c(2025, 12))
  sector_rev_2025 <- vector("list", length(sectors))
  sector_rev_2026 <- vector("list", length(sectors))
  sector_low_2026 <- vector("list", length(sectors))
  sector_up_2026 <- vector("list", length(sectors))
  for (i in seq_along(sectors)) {
    s <- sectors[i]
    sales_col <- paste0(s, "_Sales")
    price_col <- paste0(s, "_Price")
    sales_df <- clean_monthly_series(df_sub, sales_col)
    price_df <- clean_monthly_series(df_sub, price_col)
    sales_ts <- make_ts_clean(sales_df, "value")
    price_ts <- make_ts_clean(price_df, "value")
    sales_fit <- fit_one_family(sales_ts, family = family)
    price_fit <- fit_one_family(price_ts, family = family)
    # point forecasts
    sector_rev_2025[[i]] <- sales_fit$forecast_2025 * price_fit$forecast_2025 / 100
    sector_rev_2026[[i]] <- sales_fit$forecast_2026 * price_fit$forecast_2026 / 100
    # approximate lower/upper by multiplying lower/lower and upper/upper
    sector_low_2026[[i]] <- sales_fit$lower_2026 * price_fit$lower_2026 / 100
    sector_up_2026[[i]] <- sales_fit$upper_2026 * price_fit$upper_2026 / 100
  }
  agg_2025 <- Reduce(`+`, sector_rev_2025)
  agg_2026 <- Reduce(`+`, sector_rev_2026)
  interval_res <- combine_component_intervals(
    mean_list = sector_rev_2026,
    lower_list = sector_low_2026,
    upper_list = sector_up_2026
  )
  acc <- calc_accuracy(actual_2025, agg_2025)
  tibble::tibble(
    Candidate = candidate_name,
    Strategy = "Sales and Price Then Aggregate",
    Start_Date = as.Date(start_date),
    Family = family,
    RMSE = acc$RMSE,
    MAE = acc$MAE,
    MSE = acc$MSE,
    MAPE = acc$MAPE,
    AICc = NA_real_,
    Forecast_2025 = list(agg_2025),
    Forecast_2026 = list(interval_res$mean),
    Lower_2026_95 = list(interval_res$lower),
    Upper_2026_95 = list(interval_res$upper),
    Fit_Object = list(NULL),
    Model_Description = paste0("Aggregate of sector-level sales and price ", family, "
models"),
    Lambda = NA_real_,
    Resid_Mean = NA_real_,
    Resid_SD = NA_real_,
    Ljung_Box_pvalue = NA_real_,
    Resid_ACF12 = NA_real_
  )
}

```

```

}
# 15) MASTER WRAPPER TO FIT ONE CANDIDATE WITH BOTH FAMILIES
fit_candidate_both_families <- function(df_in, candidate_name, strategy, start_date) {
  if (strategy == "total_direct") {
    ets_res <- fit_total_direct_candidate(df_in, start_date, "ETS", candidate_name)
    sarima_res <- fit_total_direct_candidate(df_in, start_date, "SARIMA",
candidate_name)
  } else if (strategy == "sector_revenue_aggregate") {
    ets_res <- fit_sector_revenue_candidate(df_in, start_date, "ETS",
candidate_name)
    sarima_res <- fit_sector_revenue_candidate(df_in, start_date, "SARIMA",
candidate_name)
  } else if (strategy == "sales_price_aggregate") {
    ets_res <- fit_sales_price_candidate(df_in, start_date, "ETS", candidate_name)
    sarima_res <- fit_sales_price_candidate(df_in, start_date, "SARIMA",
candidate_name)
  } else {
    stop(paste("Unknown strategy:", strategy))
  }
  dplyr::bind_rows(ets_res, sarima_res)
}
# 16) RUN ALL 10 CANDIDATES
cat("\n===== CANDIDATE DATE COVERAGE CHECK =====\n")
for (i in seq_len(nrow(candidate_definitions))) {
  cat("\nCandidate:", candidate_definitions$candidate_name[i], "\n")
  cat("Start date:", as.character(candidate_definitions$start_date[i]), "\n")
  cat("Families:", candidate_definitions$Families_Testeds[i], "\n")
}
all_candidate_results <- purrr::pmap_dfr(
  list(
    candidate_name = candidate_definitions$candidate_name,
    strategy = candidate_definitions$strategy,
    start_date = candidate_definitions$start_date
  ),
  function(candidate_name, strategy, start_date) {
    fit_candidate_both_families(
      df_in = data_fl,
      candidate_name = candidate_name,
      strategy = strategy,
      start_date = start_date
    )
  }
)
# 17) ADD BENCHMARK MODEL
total_full_df_benchmark <- clean_monthly_series(data_fl, "Total_Revenue_Calc")
ts_total_full_benchmark <- make_ts_clean(total_full_df_benchmark, "value")
benchmark_result <- fit_benchmark_candidate(ts_total_full_benchmark)
all_candidate_results <- dplyr::bind_rows(all_candidate_results, benchmark_result)
# 18) ADD WEIGHTED SCORE WITHIN EACH CANDIDATE
all_candidate_results <- all_candidate_results %>%
  dplyr::group_by(Candidate) %>%
  dplyr::group_modify(~ add_weighted_score(.x)) %>%
  dplyr::ungroup()
# 19) FULL ACCURACY TABLE
model_accuracy_results <- all_candidate_results %>%
  dplyr::select(
    Candidate, Strategy, Start_Date, Family, Model_Description, Lambda,
    RMSE, MAE, MSE, MAPE, AICc, weighted_score,
    Resid_Mean, Resid_SD, Ljung_Box_pvalue, Resid_ACF12
  ) %>%
  dplyr::arrange(weighted_score, RMSE, MAE, MAPE)
print(model_accuracy_results)
# 20) BEST FAMILY WITHIN EACH CANDIDATE

```

```

best_within_candidate <- all_candidate_results %>%
  dplyr::group_by(Candidate) %>%
  dplyr::arrange(weighted_score, RMSE, MAE, MAPE, .by_group = TRUE) %>%
  dplyr::slice(1) %>%
  dplyr::ungroup() %>%
  dplyr::select(
    Candidate, Strategy, Start_Date, Family, Model_Description, Lambda,
    RMSE, MAE, MSE, MAPE, AICc, weighted_score,
    Resid_Mean, Resid_SD, Ljung_Box_pvalue, Resid_ACF12,
    Forecast_2025, Forecast_2026, Lower_2026_95, Upper_2026_95, Fit_Object
  )
print(best_within_candidate %>%
  dplyr::select(
    Candidate, Strategy, Start_Date, Family, Model_Description,
    RMSE, MAE, MSE, MAPE, weighted_score
  ))
# 21) ABSOLUTE BEST MODEL OVERALL
best_overall_model <- best_within_candidate %>%
  dplyr::arrange(weighted_score, RMSE, MAE, MAPE) %>%
  dplyr::slice(1)
cat("\n===== ABSOLUTE BEST MODEL OVERALL =====\n")
print(best_overall_model %>%
  dplyr::select(
    Candidate, Strategy, Start_Date, Family, Model_Description,
    RMSE, MAE, MSE, MAPE, weighted_score
  ))
# 22) 2026 FORECAST TABLE FOR BEST VERSION OF EACH CANDIDATE
forecast_2026_results <- purrr::pmap_dfr(
  list(
    Candidate = best_within_candidate$Candidate,
    Strategy = best_within_candidate$Strategy,
    Start_Date = best_within_candidate$Start_Date,
    Selected_Family = best_within_candidate$Family,
    Forecast_2026 = best_within_candidate$Forecast_2026,
    Lower_2026_95 = best_within_candidate$Lower_2026_95,
    Upper_2026_95 = best_within_candidate$Upper_2026_95
  ),
  function(Candidate, Strategy, Start_Date, Selected_Family, Forecast_2026,
    Lower_2026_95, Upper_2026_95) {
    tibble::tibble(
      Candidate = Candidate,
      Strategy = Strategy,
      Start_Date = Start_Date,
      Selected_Family = Selected_Family,
      Date = seq.Date(as.Date("2026-01-01"), by = "month", length.out = 12),
      Forecast_Revenue_Thousand = as.numeric(Forecast_2026),
      Lower_2026_95_Thousand = as.numeric(Lower_2026_95),
      Upper_2026_95_Thousand = as.numeric(Upper_2026_95)
    )
  }
)
print(forecast_2026_results)
# 23) IMPROVED FULL-HISTORY PLOTS FOR EACH CANDIDATE
full_history_plot_df <- data_fl %>%
  dplyr::select(Date, Total_Revenue_Calc) %>%
  dplyr::rename(value = Total_Revenue_Calc) %>%
  dplyr::filter(Date >= as.Date("1990-01-01"),
    Date <= as.Date("2025-12-01")) %>%
  dplyr::arrange(Date) %>%
  dplyr::filter(!is.na(value))
plot_list <- purrr::pmap(
  list(
    forecast_2026 = best_within_candidate$Forecast_2026,

```

```

    lower_2026      = best_within_candidate$Lower_2026_95,
    upper_2026      = best_within_candidate$Upper_2026_95,
    candidate       = best_within_candidate$Candidate,
    family          = best_within_candidate$Family
  ),
  function(forecast_2026, lower_2026, upper_2026, candidate, family) {
    plot_candidate_forecast(
      full_df       = full_history_plot_df,
      forecast_2026 = forecast_2026,
      lower_2026    = lower_2026,
      upper_2026    = upper_2026,
      title_text    = paste0(candidate, " (Selected Family: ", family, ")")
    )
  }
)
for (p in plot_list) {
  print(p)
}
# 24) HOLDOUT COMPARISON PLOTS FOR EACH WINNING CANDIDATE
actual_2025_total <- full_history_plot_df %>%
  dplyr::filter(Date >= as.Date("2025-01-01"),
                Date <= as.Date("2025-12-01")) %>%
  dplyr::pull(value)
holdout_plot_list <- purrr::pmap(
  list(
    predicted_2025 = best_within_candidate$Forecast_2025,
    candidate      = best_within_candidate$Candidate,
    family         = best_within_candidate$Family
  ),
  function(predicted_2025, candidate, family) {
    plot_holdout_comparison(
      actual_2025 = actual_2025_total,
      predicted_2025 = predicted_2025,
      candidate     = candidate,
      family        = family
    )
  }
)
for (p in holdout_plot_list) {
  print(p)
}
# 25) RESIDUAL DIAGNOSTICS FOR BEST OVERALL DIRECT MODEL
if (best_overall_model$Strategy == "Total Revenue Direct" ||
    best_overall_model$Strategy == "Benchmark") {
  cat("\n===== RESIDUAL DIAGNOSTICS FOR BEST OVERALL DIRECT/BENCHMARK MODEL
=====\\n")
  print(best_overall_model$Candidate)
  print(best_overall_model$Family)

  if (!is.null(best_overall_model$Fit_Object[[1]])) {
    forecast::checkresiduals(best_overall_model$Fit_Object[[1]])
  } else {
    cat("\nNo single fit object available for this model.\\n")
  }
} else {
  cat("\nBest overall model is an aggregate model, so a single residual diagnostic is
not directly comparable.\\n")
}
# 26) CANDIDATE SUMMARY TABLE FOR REPORT
candidate_summary_table <- candidate_definitions %>%
  dplyr::select(candidate_id, candidate_name, strategy, start_date, Families_Testetd)
%>%
  dplyr::rename(

```

```

    Candidate_ID = candidate_id,
    Candidate = candidate_name,
    Strategy = strategy,
    Start_Date = start_date
  )
print(candidate_summary_table)
# 27) LIMITATIONS / DIAGNOSTIC SUMMARY FOR REPORT WRITING
n_missing_total <- sum(is.na(data_fl$Total_Revenue_Calc))
interpolation_used <- any(is.na(total_full_df_benchmark$value))
limitations_summary <- tibble::tibble(
  Item = c(
    "Holdout design",
    "Forecast horizon",
    "Missing values in total revenue series",
    "Interpolation used before modeling",
    "Aggregate model intervals",
    "Benchmark included",
    "Residual diagnostics available for direct models"
  ),
  Value = c(
    "2025 reserved as out-of-sample holdout",
    "24 months from end of 2024; only 2026 reported as forecast",
    as.character(n_missing_total),
    ifelse(interpolation_used, "Yes", "No"),
    "Approximate for aggregate models",
    "Yes - seasonal naive",
    "Yes"
  )
)
cat("\n===== LIMITATIONS / DIAGNOSTIC SUMMARY =====\n")
print(limitations_summary)
# 28) OPTIONAL COMPARISON TABLE OF ALL WINNERS
selected_model_results <- best_within_candidate %>%
  dplyr::select(
    Candidate, Strategy, Start_Date, Family, Model_Description, Lambda,
    RMSE, MAE, MSE, MAPE, AICc, weighted_score,
    Resid_Mean, Resid_SD, Ljung_Box_pvalue, Resid_ACF12
  ) %>%
  dplyr::arrange(weighted_score, RMSE, MAE, MAPE)
print(selected_model_results)
# 29) EXPORT RESULTS
dir.create("report_outputs", showWarnings = FALSE)
utils::write.csv(model_accuracy_results, "report_outputs/model_accuracy_results.csv",
  row.names = FALSE)
utils::write.csv(selected_model_results, "report_outputs/selected_model_results.csv",
  row.names = FALSE)
utils::write.csv(forecast_2026_results, "report_outputs/forecast_2026_results.csv",
  row.names = FALSE)
utils::write.csv(candidate_summary_table,
  "report_outputs/candidate_summary_table.csv", row.names = FALSE)
utils::write.csv(revenue_validation_results,
  "report_outputs/revenue_validation_results.csv", row.names = FALSE)
utils::write.csv(limitations_summary, "report_outputs/limitations_summary.csv",
  row.names = FALSE)
utils::write.csv(summary_stats_total, "report_outputs/descriptive_statistics.csv", row.names = FALSE)
ggsave("report_outputs/total_revenue_series.png", width = 10,
  height = 6,
  dpi = 300)
ggsave("report_outputs/seasonal_plot.png", width = 10,
  height = 6,
  dpi = 300)
ggsave("report_outputs/stl_decomposition.png", width = 10,

```

```

        height = 6,
        dpi = 300)
for (i in seq_along(plot_list)) {
  plot_name <- gsub("[^A-Za-z0-9]+", "_", best_within_candidate$Candidate[i])

  ggplot2::ggsave(
    filename = paste0("report_outputs/", plot_name, "_forecast_plot.png"),
    plot = plot_list[[i]],
    width = 10,
    height = 6,
    dpi = 300
  )
}
utils::write.csv(summary_stats_total,
  "report_outputs/descriptive_statistics_wide.csv",
  row.names = FALSE)
utils::write.csv(descriptive_statistics_table,
  "report_outputs/descriptive_statistics.csv",
  row.names = FALSE)
# FINAL PLOT: TOTAL REVENUE OVER ENTIRE HISTORY
total_revenue_plot <- ggplot(data_fl, aes(x = Date, y = Total_Revenue)) +

  geom_line(color = "black", linewidth = 0.9) +
  labs(
    title = "Florida Monthly Electricity Revenue (Full Historical Series)",
    subtitle = "Total Electricity Revenue Across All Sectors",
    x = "Year",
    y = "Revenue (thousand $)"
  ) +
  scale_y_continuous(labels = scales::comma) +

  scale_x_date(
    date_breaks = "2 years", # ensures every year appears
    date_labels = "%Y"
  ) +
  theme_minimal(base_size = 14) +

  theme(
    plot.title = element_text(face = "bold"),
    plot.subtitle = element_text(size = 12),
    axis.title = element_text(face = "bold"),
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
print(total_revenue_plot)
ggsave(
  "report_outputs/total_revenue_series.png",
  plot = total_revenue_plot,
  width = 11,
  height = 5,
  dpi = 300
)
# FINAL DIAGNOSTIC PLOTS FOR REPORT
# - Seasonal plot
# - STL decomposition
# Make sure output folder exists
dir.create("report_outputs", showWarnings = FALSE)
# Build a clean monthly series for total revenue
total_full_df_for_plots <- data_fl %>%
  dplyr::select(Date, Total_Revenue) %>%
  dplyr::rename(value = Total_Revenue) %>%
  dplyr::arrange(Date) %>%
  dplyr::filter(!is.na(value))
ts_total_full_for_plots <- ts(

```



```

total_full_df_for_plots$value,
start = c(
  lubridate::year(min(total_full_df_for_plots$Date)),
  lubridate::month(min(total_full_df_for_plots$Date))
),
frequency = 12
)
# 1) SEASONAL PLOT
# SEASONAL PLOT FOR REPORT (IMPROVED FORMATTING
seasonal_plot_obj <- forecast::ggseasonplot(ts_total_full_for_plots) +
  ggplot2::labs(
    title = "Seasonal Plot: Florida Total Electricity Revenue",
    x = "Month",
    y = "Revenue (thousand $)"
  ) +
  # Prevent scientific notation
  ggplot2::scale_y_continuous(labels = scales::comma) +

  ggplot2::theme_minimal(base_size = 13) +
  ggplot2::theme(
    plot.title = ggplot2::element_text(face = "bold"),
    axis.title = ggplot2::element_text(face = "bold"),

    # Slightly increase right margin so legend is not clipped
    plot.margin = ggplot2::margin(10, 60, 10, 10)
  )
print(seasonal_plot_obj)
ggplot2::ggsave(
  filename = "report_outputs/seasonal_plot.png",
  plot = seasonal_plot_obj,
  width = 12, # slightly wider so legend fits
  height = 6,
  dpi = 300
)
# ACF AND PACF OF RESIDUALS FOR BEST OVERALL MODEL
cat("\n===== ACF AND PACF OF RESIDUALS: BEST OVERALL MODEL =====\n")
print(
  best_overall_model %>%
  dplyr::select(
    Candidate, Strategy, Family, Model_Description,
    RMSE, MAE, MSE, MAPE
  )
)
best_fit_object <- best_overall_model$Fit_Object[[1]]
if (!is.null(best_fit_object)) {

  resid_vec <- as.numeric(stats::residuals(best_fit_object))
  resid_vec <- stats::na.omit(resid_vec)
  resid_ts <- stats::ts(resid_vec, frequency = 12)
  p_acf <- forecast::ggAcf(resid_ts) +
    ggplot2::labs(
      title = paste0("ACF of Residuals: ", best_overall_model$Candidate),
      x = "Lag",
      y = "ACF"
    ) +
    ggplot2::theme_minimal(base_size = 12)
  p_pacf <- forecast::ggPacf(resid_ts) +
    ggplot2::labs(
      title = paste0("PACF of Residuals: ", best_overall_model$Candidate),
      x = "Lag",
      y = "PACF"
    ) +
    ggplot2::theme_minimal(base_size = 12)

```

```

print(p_acf)
print(p_pacf)
} else {
  cat("\n\nThe best overall model does not have a single stored fit object.\n")
  cat("Residual ACF/PACF is not directly available.\n")
}

```

Appendix C – Key Console Outputs

```

print(summary_stats_total)
# A tibble: 1 × 7
      Mean      Median      SD      Min      Q1      Q3      Max
  <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>    <dbl>
1 2080668. 1978575. 464931. 1174126. 1758344. 2378091. 3618351.
===== STATIONARITY TESTS (DIAGNOSTIC SERIES) =====
> print(tseries::adf.test(ts_total_for_tests))
      Augmented Dickey-Fuller Test
data:  ts_total_for_tests
Dickey-Fuller = -5.3022, Lag order = 7, p-value = 0.01
alternative hypothesis: stationary
> print(tseries::kpss.test(ts_total_for_tests))
      KPSS Test for Level Stationarity
data:  ts_total_for_tests
KPSS Level = 5.2058, Truncation lag parameter = 5, p-value = 0.01
> cat("\nSuggested regular differences:", forecast::ndiffs(ts_total_for_tests), "\n")
Suggested regular differences: 1
> cat("Suggested seasonal differences:", forecast::nsdiffs(ts_total_for_tests), "\n")
Suggested seasonal differences: 1
> revenue_validation_results <- purrr::map_dfr(validation_sectors, function(s) {
+   if (s == "Total") {
+     reported_col <- "Total_Revenue"
+   }
+   print(revenue_validation_results)
+ })
# A tibble: 5 × 6
  Sector      Mean_Abs_Diff Max_Abs_Diff Mean_Pct_Diff Max_Pct_Diff N_Compared
  <chr>          <dbl>          <dbl>          <dbl>          <dbl>          <int>
1 Residential      227.           646.           0.0249         0.0642           432
2 Commercial       173.           449.           0.0302         0.0783           432
3 Industrial        37.0           88.2           0.0395         0.103            432
4 Transportation    0.171          0.395           0.0269         0.0862           276
5 Total            312.          1116.           0.0152         0.0475           276
> print(model_accuracy_results)
# A tibble: 21 × 16
  Candidate Strategy Start_Date Family Model_Description Lambda RMSE MAE
MSE MAPE AICc weighted_score
  <chr> <chr> <date> <chr> <chr>
  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 Total Direc... Total R... 2010-01-01 SARIMA ARIMA(3,1,0) (2,1... 4.10e-5 1.13e5 9.34e4
1.27e10 3.20 -532. 0
2 Total Direc... Total R... 2010-01-01 SARIMA ARIMA(3,1,0) (2,1... 4.10e-5 1.13e5 9.34e4
1.27e10 3.20 -532. 0
3 Total Direc... Total R... 2015-01-01 SARIMA ARIMA(3,1,0) (2,1... 4.10e-5 1.18e5 9.69e4
1.40e10 3.29 -320. 0
4 Total Direc... Total R... 2005-01-01 SARIMA ARIMA(3,0,0) (2,1... 4.10e-5 1.30e5 1.07e5
1.69e10 3.61 -740. 0
5 Sector Reve... Sector ... 2015-01-01 SARIMA Aggregate of sec... NA 1.41e5 1.15e5
1.98e10 3.83 NA 0
6 Sector Reve... Sector ... 2010-01-01 SARIMA Aggregate of sec... NA 2.16e5 1.87e5
4.65e10 6.26 NA 0
7 Sector Reve... Sector ... 2010-01-01 SARIMA Aggregate of sec... NA 2.16e5 1.87e5
4.65e10 6.26 NA 0
8 Sales x Pri... Sales a... 2010-01-01 ETS Aggregate of sec... NA 2.27e5 2.06e5
5.15e10 6.95 NA 0

```

```

 9 Sales x Pri... Sales a... 2010-01-01 ETS      Aggregate of sec... NA      2.27e5 2.06e5
5.15e10  6.95    NA              0
10 Benchmark S... Benchma... 1990-01-01 SNAIVE Seasonal naive    NA      2.30e5 2.04e5
5.30e10  6.80    NA              0
# i 11 more rows
# i 4 more variables: Resid_Mean <dbl>, Resid_SD <dbl>, Ljung_Box_pvalue <dbl>,
Resid_ACF12 <dbl>
# i Use `print(n = ...)` to see more rows
> print(best_within_candidate %>%
+       dplyr::select(
+         Candidate, Strategy, Start_Date, Family, Model_Description,
+         RMSE, M .... [TRUNCATED])
# A tibble: 11 x 10
  Candidate          Strategy Start_Date Family Model_Description    RMSE
MAE      MSE  MAPE weighted_score
<chr>      <dbl> <dbl>      <chr>      <date>      <chr> <chr>      <dbl>
<dbl>      <dbl> <dbl>      <dbl>
1 Benchmark Seasonal Naive    Benchma... 1990-01-01 SNAIVE Seasonal naive    2.30e5
2.04e5 5.30e10  6.80              0
2 Sales x Price Aggregate - ... Sales a... 2010-01-01 ETS      Aggregate of sec... 2.27e5
2.06e5 5.15e10  6.95              0
3 Sales x Price Aggregate - ... Sales a... 2010-01-01 ETS      Aggregate of sec... 2.27e5
2.06e5 5.15e10  6.95              0
4 Sector Revenue Aggregate -... Sector ... 2010-01-01 SARIMA Aggregate of sec... 2.16e5
1.87e5 4.65e10  6.26              0
5 Sector Revenue Aggregate -... Sector ... 2015-01-01 SARIMA Aggregate of sec... 1.41e5
1.15e5 1.98e10  3.83              0
6 Sector Revenue Aggregate -... Sector ... 2010-01-01 SARIMA Aggregate of sec... 2.16e5
1.87e5 4.65e10  6.26              0
7 Total Direct - 2010+        Total R... 2010-01-01 SARIMA ARIMA(3,1,0) (2,1... 1.13e5
9.34e4 1.27e10  3.20              0
8 Total Direct - Full History Total R... 1990-01-01 SARIMA ARIMA(1,0,2) (0,1... 2.50e5
2.27e5 6.24e10  7.65              0
9 Total Direct - Last 10 Yea... Total R... 2015-01-01 SARIMA ARIMA(3,1,0) (2,1... 1.18e5
9.69e4 1.40e10  3.29              0
10 Total Direct - Last 15 Yea... Total R... 2010-01-01 SARIMA ARIMA(3,1,0) (2,1... 1.13e5
9.34e4 1.27e10  3.20              0
11 Total Direct - Last 20 Yea... Total R... 2005-01-01 SARIMA ARIMA(3,0,0) (2,1... 1.30e5
1.07e5 1.69e10  3.61              0
===== ABSOLUTE BEST MODEL OVERALL =====
> print(best_overall_model %>%
+       dplyr::select(
+         Candidate, Strategy, Start_Date, Family, Model_Description,
+         RMSE, MAE, .... [TRUNCATED])
# A tibble: 1 x 10
  Candidate          Strategy          Start_Date Family Model_Description    RMSE
MAE      MSE  MAPE weighted_score
<chr>      <dbl> <dbl>      <chr>      <date>      <chr> <chr>      <dbl>
<dbl>      <dbl> <dbl>      <dbl>
1 Total Direct - 2010+ Total Revenue D... 2010-01-01 SARIMA ARIMA(3,1,0) (2,1... 1.13e5
93416. 1.27e10  3.20              0

```